Integrating Functional and Structural Semantics for Web API Recommendation via Multi Meta-Path Aggregation

Chunyan Sang, Yahao Liu, Shigen Liao, and Junhao Wen

***Abstract*—With the increasing popularity of Internet services, developers can reuse or mix these services to create new applications (such as mashups) to improve development efficiency. In previous studies, several web API recommendation methods have been proposed, some of which rely on mashup demand information to extract necessary functional features. These methods often incorporate Quality of Service (QoS) assessments of network services to enhance recommendations. Others utilize graph neural networks to extract implicit structural semantics from the web API network and apply them to build downstream tasks. Despite the significant contributions of these methods in advancing web API recommendations, the challenge lies in how to better extract structural information in highly sparse conditions and combine it with functional information to optimize recommendation performance. In this paper, we construct a mashup-tag-API graph based on tag similarity to address the sparsity between mashups and web APIs. We also design a method to jointly incorporate structural and functional information. Structural information is mined through multiple meta-path aggregation mechanisms. Mashups are aggregated into different clusters based on structural information. Developers' requirement features are then extracted using functional semantic components and mapped to groups of similar mashups. Finally, the model recommends highly compatible network APIs based on historical call records from these groups. Comprehensive testing using real-world datasets from ProgrammableWeb demonstrates that our method outperforms baseline methods in terms of accuracy, recall, and MAP.**

***Index Terms*—Service recommendation, recommender system, mashup development, network embedding.**

I. Introduction

With the development of Internet related technologies, the number and variety of network services now offered have risen considerably. Consequently, Internet has also entered the era of Web 2.0. Mashups are an indication of Web 2.0 and a mix of web API services, which have garnered considerable interest from both industry and academia. Using content retrieved from other API services, developers can create these applications. Unlike traditional integration technologies, developers can quickly and easily combine Internet API services using mashups. This approach allows developers to flexibly utilize API services and simplifies development tasks [1]. As a result, several online mashups and web API repositories, including ProgrammableWeb, have been created. These repositories provide great convenience for developers. Numerous web APIs are available to support external calls. Developers can create mashups by combining these web API services with various functions, which can meet user needs [2]. Therefore, developers can quickly select appropriate web APIs to build mashups based on their specific needs, greatly improving development efficiency. In a real-world mashup-API network, there are many objects and interconnections between mashups and the API services. For example, as of April 2021, ProgrammableWeb had over 23000 web APIs and over 7000 mashups, making it one of the largest web API directories. However, the rapid increase in the number of web APIs has made it extremely difficult to manually select the required web APIs from such large databases. Taking the mashup design of "city Walker" as an example, it needs to integrate multiple functions such as location discovery, detail evaluation, location navigation, and user generated content management. The available APIs (such as maps, reviews, social media) often have inconsistent data formats, quality, update frequency, and authorization methods. Naturally, the method for recommending suitable web APIs to developers have received significant attention.

In the past few years, many web API recommendation methods have been proposed, including content-based [3], [4], [5], [6], [7], QoS-based [8], [9], [10], [11], and hybrid methods based on deep neural networks [12], [13], [14], [15], [16]. Among them, methods based on graph neural networks are widely used due to their powerful representation learning capabilities [17], [18]. They can not only represent the functional relationships between mashups and web APIs, but also capture higher-order semantic associations from the interaction structure. Zhang et al. [19] constructed an API connection graph based on historical interaction records of APIs. They then used GCN for feature aggregation and leveraged structural information to assist in recommendations. Yu et al. [15] utilized GraphGAN to mine the implicit relationships between mashups and APIs by construct-

Received 15 April 2024; revised 15 July 2025; accepted 11 August 2025. Date of publication 15 August 2025; date of current version 9 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62002037 and in part by the research project of Chongqing CSTC under Grant cstc2019jcyj-msxmX0588. *(Corresponding author: Chunyan Sang.)*

Chunyan Sang, Yahao Liu, and Shigen Liao are with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: sangcy@cqupt.edu.cn).

Junhao Wen is with the School of Big Data & Software Engineering, ChongQing University, University City South Road, Chongqing 401331, China.

Digital Object Identifier 10.1109/TSC.2025.3599574

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

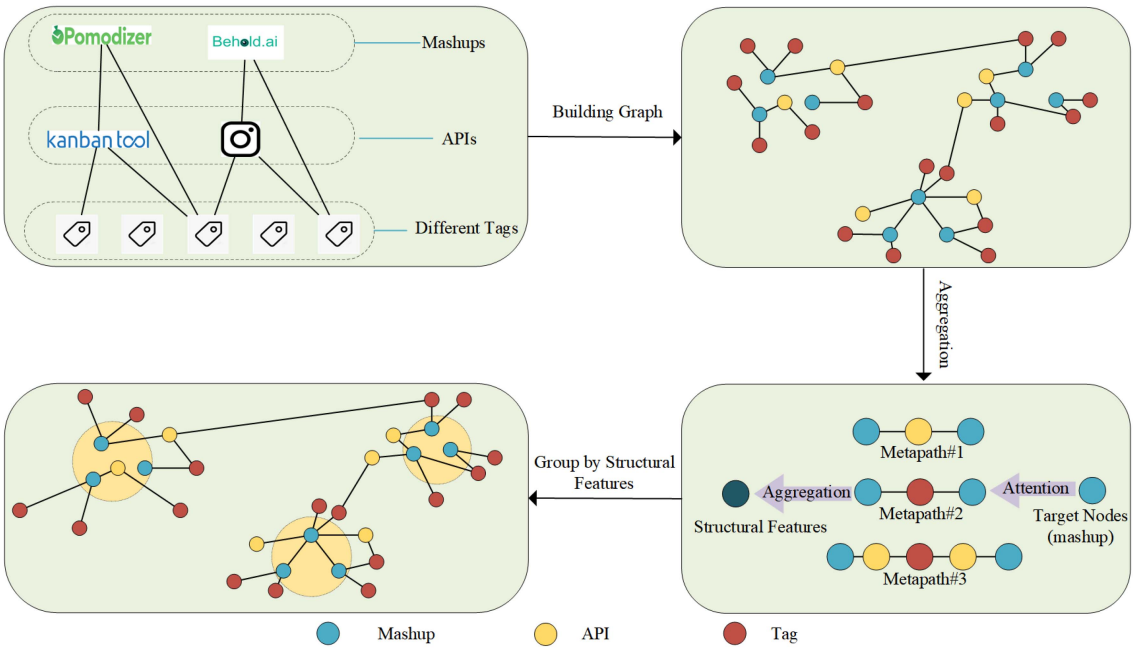


Fig. 1. Structural feature extraction and group.

ing a mashup-API graph, which enhanced recommendations.

The interaction between web APIs and mashups is severe sparsity. Existing approaches rely on simple graph structures and aggregation methods are often unduly influenced by nodes with a higher number of interactions. To address this, Hu et al. [20] proposed a web API recommendation method based on a biased random walk, which utilized the functional similarity of mashup nodes and the historical interactions between web APIs and mashups to guide the random walk. However, a heterogeneous graph constructed solely from historical interaction records primarily captures associations between nodes, making the extraction of structural information potentially insufficient in highly sparse environments. Additionally, random walks struggle to capture comprehensive structural features across the entire graph and face significant computational overhead due to the requirement for a high number of walks. To overcome these limitations, we propose a web API recommendation method based on multi-path aggregation. Our method first leverages similar functional tags to link related mashups and web APIs, thereby alleviating the sparsity of mashup-API interactions. Subsequently, we aggregate features from multiple meta-paths to extract the structural features of the mashup-API-tag heterogeneous graph. This aggregation strategy addresses the performance consumption associated with multiple random walks.

In sum, this paper studies a web API recommendation approach by integrating functional and structural semantics through a multi-path aggregation method. First, considering the functional label similarity between web APIs and mashups, a mashup-tag-API heterogeneous graph is constructed to alleviate the sparsity of interactions between them. Then, a multi-path aggregation method is employed to mine the structural semantics of the heterogeneous graph, which can modulate the influence of different nodes on the central node based on distinct meta-paths. Correspondingly, mashups with similar invocation preferences are grouped into the same cluster, as illustrated in Fig. 1. Finally, specific mashup clusters are identified based on the developers' functional requirements, leveraging the historical interactions of web APIs within these clusters for the final recommendations. Experiments are conducted on the real-world ProgrammableWeb dataset, and the results indicate that the proposed method outperforms six baseline models in terms of Precision, Recall, and Mean Average Precision (MAP).

The remainder of this article is organized as follows: Section II reviews related work. Section III elaborates on the problem definition and related techniques. Section IV provides a detailed introduction to our proposed framework. The experimental results and hyperparameters settings are given in Section V. In Section VI, we summarize this work and discuss future research directions.

II. Related Works

A. Web API Recommendation

In recent years, web API recommendation has received widespread attention from academia and industry. Many web API recommendation methods have been proposed, which are generally divided into three categories, content-based, QoS-based, and graph-based method.

The content-based service recommendations primarily examine the contextual similarity between user needs and services, which is typically gauged through a service's functional description. Qi et al. [5] developed a text description-driven and compatibility-aware web API recommendation technique that ensured compatibility between recommended APIs by utilizing historical mashup records. Their approach allowed software

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

developers to submit a straightforward text document describing the desired mashup functionality. It then used text description mining to precisely capture developer requirements and returned a set of compatible APIs. Gao et al. [21] paradigm where each service was treated as a document, with its co-occurring services acting as the document's words. To address recommendation challenges and enhance suggestion diversity, Yao et al. [6] introduced a probability matrix decomposition method with implicit correlation regularization. QoS-based approaches are frequently combined with content-based methods. Wu et al. [22] presented a method that determines user-object similarity through direct comparison of their property values. Chen et al. [8] introduced a neighborhood-integrated matrix factorization model, which combines model-based and neighborhood-based collaborative filtering (CF) approaches.

However, most of these methods only utilize a single type of content information (such as descriptions) and ignore the structural information from the mashup-API-tag network. It is necessary to find elements that can fit the mashup being composed from both a structural and a content point of view when recommending web APIs [3]. Deep neural networks can learn the nonlinear interactions between mashups and API services and have strong feature representation capabilities [18].

Recently, some researchers have begun to leverage deep neural networks to mine the structural information within service networks to explore the implicit relationship between API services and mashups. Typically, they extract features from content information to serve as node features for mashups and API services. Then, using historical interactions and other attributes, they construct a service network. This network is embedded via deep neural networks to obtain enhanced representation vectors. Xie et al. [18] proposed a GAN-based service recommendation method for mashup creation. By sampling metapaths representing different semantic linkages, they computed metapath-based similarity metrics to generate a similarity matrix across mashups and services. Subsequently, through adversarial training between the discriminator and generator, the discriminator guides the generator to produce preference vectors for developers. After that, Yu et al. [15] also employed GANs for feature enhancement. To better extract content information, they integrated BERT into their framework. Liang et al. [17] proposed a method based on metapath similarity. These studies collectively demonstrate the effectiveness of metapath-based approaches in service networks. A graph representation learning method-based API package recommendation model was put forth by Qi et al. [13]. Zhang et al. [19] proposed an API link prediction approach based on GCN and Doc2vec. Wang et al. [23] proposed a knowledge graph-based random walk method to learn low dimensional embedding representations of mashups and web services, thereby improving the accuracy of recommendations. These methods all rely on the powerful representation learning capabilities of deep neural networks, which aggregate neighboring information in diverse ways to uncover hidden structural information. However, most of them utilize relatively simple network structures and a single feature aggregation strategy to derive the mashup's representation vector, subsequently integrating it directly with functional features for use. Inspired by these efforts, we have designed a method based on FSSIM to combine functional and structural information.

B. Recommendation Method Based on Heterogeneous Graphs

Graph neural networks (GNNs) are widely used in various recommendation tasks due to their strong feature representation capabilities. Many real-world scenarios can be modeled as heterogeneous graph, and leveraging structural information within these graphs enhances the performance of recommendation systems [24], [25]. Concurrently, selecting specific original metapaths in heterogeneous graphs to enhance the aggregation of certain structural semantic information has also been validated as effective [26], [27]. Hu et al. [20] designed a biased random walk strategy for web API recommendation within heterogeneous graphs. They first constructed a heterogeneous network to model diverse connection types. Subsequently, they devised a strategy based on neighboring node types to guide the biased random walks, aiming to capture the diversity of higher-order connectivity patterns. This approach, similar to the metapath-based methods, was designed to enhance feature aggregation along specific paths. Wang et al. [28] proposed a semantic-enhanced heterogeneous graph network for mashup creation to improve structural information mining. Li et al. [29] introduced knowledge graphs as auxiliary information for web service recommendations to mitigate matrix sparsity and cold start issues. Similar to conventional heterogeneous graphs approach, they also utilized metapath strategies with the knowledge graph to enhance the model's capability. However, these methods often overlook the adverse effects caused by inherently strong data sparsity. Some approaches combat sparsity by incorporating multi-source information, such as using labels as attribute to establish implicit connections [15], [20]. Unlike these approaches, we directly use similar tags as entity nodes for diagramming, which directly enhances the relationship between mashups and web APIs that have related functionalities, thereby alleviating the sparsity issue to some extent.

III. Problem Definition & Preliminaries

A. Problem Description

In this article, we build a heterogeneous graph of mashup-service-tags, mashups, services, and tags are three different types of nodes in this graph. The graph includes edges between mashups and APIs they invoke, as well as edges connecting mashups/APIs to their associated tags. The web API recommendation problem is defined as follows: Given the mashup-API composition relationships and relevant side data. When provided with a new mashup m' (which lacks API call records) and its functional description des'_m, the goal is to identify the top-k services that best fulfill the requirements described in des'_m of m'.

Definition 1 (web API): Each API service has a description and a set of tags. For service s_i we denote its description as

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

$des_{i,j}$, where i represents the unique identifier of the API. Thus, the set of descriptions for all API services is defined as: $Des_s = \{des_{s,1}, des_{s,2}, \mathrm{des}_{s,3}, \ldots, des_{s,N_a}\}$, where N_a represents the total number of API services. Similarly, service s_i has a set of tags, denoted as $\mathrm{tag}_{m,i} = \{\mathrm{tag}_i, \mathrm{tag}_j, \ldots, \mathrm{tag}_n\}$. Each tag ($\mathrm{tag}_i$, tag_j, etc.) is a specific label with a unique identifier. The collection of tag sets for all API services is then: $\mathrm{Tag}_s = \{\mathrm{tag}_{s,1}, \mathrm{tag}_{s,2}, \ldots, \mathrm{tag}_{s,N_a}\}$.

In our method, we traverse the functional description of the web API and encode it into a text feature vector using BERT. When constructing the heterogeneous mashup-tag-API graph, tags are treated as special nodes. An edge exists between an API node and a tag node if the API's tags contain that tag.

Definition 2 (mashups): A mashup m_i has a description denoted as $des_{m,i}$,where i represents the id of the mashup. Each API service also has a unique id. So we have a set of mashup descriptions $\mathrm{Des}_m = \{des_{m,1}, \mathrm{des}_{m,2}, \mathrm{des}_{m,3}, \ldots, \mathrm{des}_{m,N_m}\}$, where N_m represents the number of mashups. Each mashup m_i has an associated set of tags $\mathrm{tag}_{m,i} = \{\mathrm{tag}_i, \mathrm{tag}_j, \ldots, \mathrm{tag}_n\}$. The collection of all mashup tag sets is $Tag_m = \{tag_{m,1}, tag_{m,2}, \ldots, tag_{m,N_m}\}$. Specifically, a mashup m_i invokes a collection of API services $M_{i,s} = \{s_i, s_j, \ldots, s_k\}$. Here, s_i, s_j and s_k each represent a specific API service within this collection.

Similarly, we traverse the functional descriptions of mashups and encode them into functional feature vectors using BERT. Concurrently, we extract the tag nodes associated with mashups, which correspond to or are similar to those of web APIs, and incorporate these tags as a distinct node type. This approach establishes edges between both mashups and APIs sharing the same tag node, thereby enhancing graph connectivity. This optimization proves particularly beneficial for improving model performance on web API datasets exhibiting a significant long-tail distribution.

Definition 3 (heterogeneous information network): A heterogeneous graph is defined as a graph $G = (V, E)$. The set of node types is denoted by N, and the number of distinct node types is denoted as $|N|$. Similarly, a heterogeneous graph contains different types of edges. The set of edge types is denoted by L, and the number of distinct edge types is denoted as $|L|$. A graph is heterogeneous if it satisfies the condition $|L|+|N| > 2$.

In this article, we present a heterogeneous graph with nodes representing mashup, API, and tag entities as a basis for mining structural information. By implementing a shared label strategy, the tag nodes serve as a bridge between mashup and API nodes, thereby alleviating the data sparsity.

Definition 4 (metapath): A metapath P is defined as a path in the form of $C_1 \xrightarrow{Rel_1} C_2 \xrightarrow{Rel_2} \cdots \xrightarrow{Rel_l} C_{l+1}$. It describes a composite relation $R = Rel_1 \circ Rel_2 \circ \cdots Rel_l$ between node types C_1 and $C_l + 1$, where denotes the relations composition operator. The complexity of heterogeneous information networks necessitates the use of metapaths to better capture the structural information and implicit semantics among objects within the network. We select three metapaths and perform mashup embedding based on them. Here, M represents mashup, A represents web APIs, and T represents tag:

(1) $M \to A \to M$: It indicates that mashups that choose the same API may have similar functionality features.

(2) $M \to A \to T \to A \to M$: It indicates that mashups that choose APIs with similar labels may have similar functionality features.

(3) $M \to T \to M$: It indicates that mashups with the same tag may have similar functionality features.

B. Preliminaries

1) BERT: We introduce Bidirectional Encoder Representation from Transformers (BERT) as a functional feature extraction component into our model. BERT is a pre-trained language model based on the Transformer architecture. Trained unsupervised on extensive textual corpora, BERT learns rich semantic representations and has demonstrated remarkable performance across various NLP tasks. Its breakthrough lies in effectively handling context dependency and bidirectional information flow, overcoming limitations of previous models. Leveraging the Transformer's self-attention mechanism, BERT excels at capturing intricate relationships between vocabulary items while considering the entire context. This mechanism also allows BERT to efficiently capture long-range dependencies, endowing it with outstanding performance in semantic understanding tasks. Through our research, we find that compared to traditional word vector based methods, BERT can better understands the meaning of words within different sentence contexts, thereby more effectively learning the functional features of mashups and web services.

This article designs a functional semantic component based on BERT. Its primary function is to encode the functional descriptions of nodes to prepare for subsequent mashup operations that identify similar functionalities. Additionally, it encodes developers' requirement descriptions.

2) Heterogeneous Graph Neural Network: Heterogeneous Graph Neural Networks (HGNN) have gained prominence in the field of recommender systems as an effective approach to learning low-dimensional embeddings for target nodes. Leveraging their robust representation learning capabilities, HGNNs consistently demonstrate effective performance in recommendation tasks. This effectiveness stems from their ability to adeptly handle complex structural information within recommendation systems.

However, extracting high-quality structural information from the intricate architecture of heterogeneous graph neural networks remains a challenging. To address this, researchers often define specific metapaths. These predefined pathways through the heterogeneous graph enable the focused extraction of pertinent structural information, thereby effectively capturing relevant contextual information for recommendation systems.

In this paper, we construct a heterogeneous graph neural network to learn the structural features of mashups. Compared to traditional isomorphic and bipartite graphs, heterogeneous graphs incorporate richer semantic information. Beyond utilizing mashups and web APIs as graph nodes, we additionally introduce their associated tags as distinct node types. Our research

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

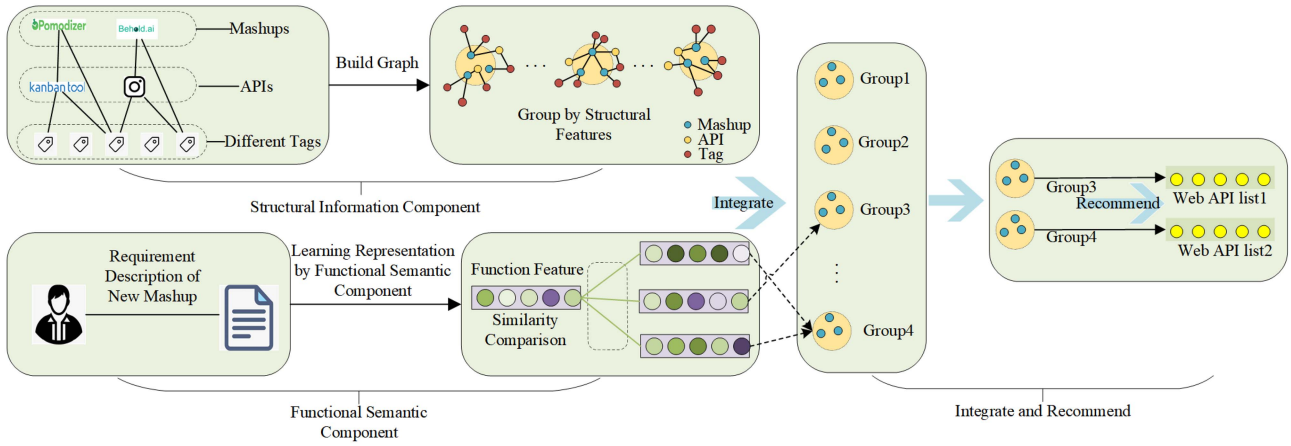


Fig. 2. The model of FSSIM for recommendation.

reveals that mashups and web APIs share a significant number of common tags. These share tags enhance graph connectivity and facilitate more effective learning of mashup preferences. To learn diverse aggregation strategies under different metapaths, we incorporate a multi-paths attention mechanism into our network [26]. This approach enables superior extraction of the structural features inherent in mashups.

IV. Approach

The proposed FSSIM model (detailed in Fig. 2) comprises three core components. Its recommendation process operates as follows: First, the functional information extraction component extracts functional information from the descriptive texts of mashups and API services. Based on the attribute information and historical invocation records of mashups and web APIs, it constructs a mashup-API-tag heterogeneous information network. This establishes the data foundation for modeling functional semantics and structural relationships. Second, the structural information mining and clustering component leverages a multi-metapath aggregation mechanism. This component mines deep structural information within the constructed heterogeneous network. Subsequently, it clusters mashups into groups based on this structural information aiming to extract and utilize structural association features among mashups. Finally, during the recommendation phase, the joint recommendation component first encodes the functional requirement descriptions provided by developers to obtain functional feature vectors. Based on these features, it identifies several functionally similar mashups from among all mashups. Significantly, it then locates these similar mashups within their respective structural clusters. Finally, it generates and recommends a list of web APIs based on the historical invocation records within the target mashup's structural cluster. This integrated approach facilitates recommendations by fist identifying relevant structural clusters through functional similarity and then leveraging collaborative invocation patterns within those clusters.

A. Functional Semantic Component

The foundation of functional semantic components is BERT. Adjustments have been made to the BERT backbone to adapt it to our specific tasks. For existing mashups, we consider a set of description $\mathrm{Des}_m = \{des_{m,1}, \mathrm{des}_{m,2}, \mathrm{des}_{m,3}, \ldots, \mathrm{des}_{m,N_m}\}$. We obtain a token sequence by segmenting each description into word. Thus, for each description, we have a token sequence as shown in (1):

$$T = [t_1, t_2, \ldots, t_n] \tag{1}$$

Next, we add special markings to obtain a new token sequence as shown in (2):

$$T' = [[\mathrm{CLS}], t_1, t_2, \ldots, t_n, [\mathrm{SEP}]]. \tag{2}$$

Next, map each token to a high-dimensional space $E \in R^{|N|\times d}$ using word embeddings and positional embeddings, where $|N|$ is the vocabulary size and d is the is the dimension of the embedding vector. The position embedding matrix is denoted P. The input to the embedding layer is given by (3):

$$\mathrm{EmbeddingLayer}(\mathrm{des}) = \{E[t_i] + P[t_i]\}_{i=1}^{n} \tag{3}$$

where $E[t_i]$ is the word embedding and $P[t_i]$ is the position embedding corresponding to position t_i. A multi-layer transformer encoder is then used. Each layer consists of a multi-head self-attention layer and a position-wise feedforward layer, as shown in (4):

$$\mathrm{vecl}_{\mathrm{des}}^{(l)} = \mathrm{TransformerLayer}\left(\mathrm{vecl}_{\mathrm{des}}^{(l-1)}\right) \tag{4}$$

where $\mathrm{vecl}_{\mathrm{des}}^{(l)}$ represents the output of the transformer at layer l, and its input is the output from layer $l-1$. We then introduce the multi-head attention layer. The input to this layer combines the embedding vector and position encoding as given in (5). Here, $E[t_i]$ and $P[t_i]$ represent the semantic embedding and positional encoding for token t_i, respectively. The seg component refers to the segmentation embedding, which distinguishes sentence membership by assigning different segment IDs (e.g.,

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

0 or 1).

$$(e, pos, seg) = (E[t_i], P[t_i], seg) \tag{5}$$

We then apply a linear transformation on each input embedding: e (semantic), pos (position), and seg (segment), as defined in (6):

$$\begin{cases} Q_i = eW_{Qi} + \text{pos}W_{Pi} + \text{seg}W_{Si} \\ K_i = eW_{Ki} + \text{pos}W_{Pi} + \text{seg}W_{Si} \\ V_i = eW_{Vi} + \text{pos}W_{Pi} + \text{seg}W_{Si} \end{cases} \tag{6}$$

Then perform multi-head attention calculation as shown in (7) and (8):

$$\text{Atten}(Q_i, K_i, V_i) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_O \tag{7}$$

$$\text{head}_i = \text{MH}(Q_i, K_i, V_i) \tag{8}$$

where $W_{Ki}, W_{Pi}, W_{Si}, W_O$ are the weights of the linear transformations. We then design the position-wise feed-forward Layer, which consists of two dense layers that applies to the last dimension, as shown in (9):

$$\text{FFN}(\text{vecl}_{\text{des}}) = \text{ReLU}\left(W_{F1} \cdot \text{ReLU}(W_{F2} \cdot \text{vecl}_{\text{des}} + b_{F1}) + b_{F2}\right) \tag{9}$$

where W_{F1}, W_{F2} are weights and b_{F1}, b_{F2} are biases. Finally, contextual representations are extracted using residual connections and layer normalization, as shown in (10):

$$\text{vec}_{\text{m}_{\text{des}}} = \text{LayerNorm}(\text{vecl}_{\text{des}} + \text{FFN}(\text{MH}(\text{vecl}_{\text{des}}))) \tag{10}$$

We use vec_{des} as functional feature representation for mashups. When a new mashup needs to be created, we identify the most similar set of existing mashups based on the proposed functional requirements. Cosine similarity is used for this calculation, a method commonly applied to process content descriptions in mashups [23], [30], as shown in (11):

$$\text{similarity}(\mathbf{vec_A}, \mathbf{vec_B}) = \frac{\mathbf{vec_A} \cdot \mathbf{vec_B}}{\|\mathbf{vec_A}\| \cdot \|\mathbf{vec_B}\|} \tag{11}$$

We select the top N mashups with the highest functional similarity as candidate mashup set, as shown in (12):

$$Mash_{set} = top_N(similarity(m, m')) \tag{12}$$

where m' and m denote the function vectors of the new mashup and all existing mashups, respectively.

B. Structural Information Component

We construct a heterogeneous service network and have utilized graph neural networks to learn node features as structural information components. There are three types of nodes in the network: mashups, web APIs, and tags. Among these, mashups serve as the target nodes. Since different node types may have unequal feature vector dimensions, we first project the features of different node types onto the same latent vector space. To achieve this, we apply specific types of linear transformations to project the feature vectors of each node type onto this common latent space, as shown in (13):

$$h'_m = W_M \cdot x_m \tag{13}$$

where the initial feature vector for node m is denoted as x_m, and h'_m represents the projected latent vector for node m. W_M is the parametric weight matrix for nodes of type M. Next, we design an aggregation layer. Through this metapath aggregation layer, the central (target) node aggregates features from its neighboring nodes along the metapath. We define a metapath instance as $P(m, u)$. Within this instance, the neighbor node connected to m is denoted as u, and the internal nodes of $P(m, u)$ are defined as $\{c^{P(m,u)}\} = P(m, u) \setminus \{u, m\}$. During internal aggregation, the features of each node within the metapath instance are transformed into vector representations $h_{P(m,u)}$ using a specific metapath instance encoder, as shown in (14).

$$\begin{aligned} \mathbf{h}_{P(m,n)} &= f_\theta\left(P(m, n)\right) \\ &= f_\theta\left(\mathbf{h}'_m, \mathbf{h}'_n, \left\{\mathbf{h}'_t, \forall t \in \left\{m^{P(m,n)}\right\}\right\}\right) \end{aligned} \tag{14}$$

Among them, h represents an encoding method defined in (15). This function first computes the element-wise mean of the node vectors along metapath instance $P(m, u)$, then concatenates this mean with a linearly transformed representation.

$$\mathbf{h}_{P(m,n)} = \mathbf{W}_P \cdot \text{MEAN}\left(\{\mathbf{h}'_t, \forall t \in P(m, n)\}\right) \tag{15}$$

Next, we design an attention layer to compute a weighted sum of metapath instances $P(m, n)$ connected to target node m. The key insight is that different metapath instances contribute to the target node's representation to varying degrees. We model this by learning a normalized importance weight α^P_{mn} for each metapath instance, followed by a weighted summation across all instances as defined in (16).

$$\begin{cases} e^P_{mn} = \text{LeakyReLU}\left(\mathbf{a}^\top_P \cdot \left[\mathbf{h}'_m \| \mathbf{h}_{P(m,n)}\right]\right) \\ \alpha^P_{mn} = \frac{\exp\left(e^P_{mn}\right)}{\sum_{s \in \mathcal{N}^P_m} \exp(e^P_{ms})} \\ \mathbf{h}^P_m = \sigma\left(\sum_{u \in \mathcal{N}^P_m} \alpha^P_{mn} \cdot \mathbf{h}_{P(m,n)}\right) \end{cases} \tag{16}$$

Here, a_P denotes the parameterized attention vector for metapath P, and $||$ represents the vector concatenation operator. N^P_m is set of neighbor nodes of m connected via metapath P. e^P_{mn} quantifies the importance metapath instance $P(m, n)$ to node m, while α^P_{mn} denotes its normalized importance weight among all metapaths. Finally, the output is transformed through an activation function $\sigma(\cdot)$. We extend this mechanism to K independent attention heads and concatenate their outputs as defined in (17).

$$\mathbf{h}^P_m = \prod_{k=1}^{K} \sigma\left(\sum_{n \in \mathcal{N}^P_m} \left[\alpha^P_{mn}\right]_k \cdot \mathbf{h}_{P(m,n)}\right) \tag{17}$$

Next, we perform the above operations on each metapath, obtaining sets of latent vectors $\{\mathbf{h}^{P_1}_{\boldsymbol{m}}, \mathbf{h}^{P_2}_{\boldsymbol{m}}, \ldots, \mathbf{h}^{P_M}_{\boldsymbol{m}}\}$. We then sum these vectors and take their average to obtain $\mathbf{h}^{P_{avg}}_m$. Finally, we apply an additional linear transformation followed by a nonlinear activation function to project the node embeddings into the vector space with the desired output dimension, as shown in (18).

$$\mathbf{h}_{\boldsymbol{m}} = \sigma\left(\mathbf{W}_{\boldsymbol{o}} \cdot \mathbf{h}^{avg}_{\boldsymbol{m}}\right) \tag{18}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

where $\sigma(\cdot)$ is an activation function, and W_o is a weight matrix, weintegrate this representation into downstream tasks.

Specifically, we frame the web API recommendation task as a multi-objective, multi classification problem. We use the APIs connected to a mashup as target labels, transforming this into a multi-label classification task. Consequently, we employ Binary Cross-Entropy Loss (BCELoss) as the loss function. For a given label i, let σx_{n_i} denote the predicted value, σy_{n_i} denote the ground-truth value, and w_{n_i} be a weighting parameter. The formal implementation is given in (19):

$$l_{n,i} = -w_{n,i}[y_{n,i} \cdot \log \sigma(x_{n,i}) + (1 - y_{n,i}) \log (1 - \sigma(x_{n,i}))] \tag{19}$$

Then we sum these per-label losses across all APIs and compute the mean value to obtain the final loss function, as defined in (20):

$$\mathrm{L} = \sum_{i=1}^{l} \frac{1}{J1} \left(\sum_{n=1}^{J1} l_{n,i} \right) \tag{20}$$

After the above tasks, we obtain the corresponding structural feature vector representations for all mashups. These mashups are then grouped based on the similarity between their feature vectors, computed using (11).

C. Integrate and Recommend

When a mashup needs to be created, we input its requirements description into the functional semantic component, ultimately obtaining the $Mash_{set}$ shown in (12). Then we manually set two natural numbers N and M. N represents the number of top elements we extract from the $Mash_{set}$. M indicates the number of groups into which the mashup will be divided based on structural information. For each group, we separately count and sort the records of calling the web API. This process results in N candidate API combinations. Since the structural information component groups mashups with similar call records and semantics together, the recommended web API has high compatibility. As shown in Algorithm 1.

Algorithm 1: Algorithm of FSSIM.

Input: Descriptions of mashups Des_m, Descriptions of services Des_s, mashup-API-tag network graph $G = (V, E)$, Description of the new mashup des'_m
Output: Preferences of m' for $top - K$ service

for each mashup m in M and each service s in S **do**
  $vec_{m_{des}} \leftarrow$ BERT(m_{des})
  $vec_{s_{des}} \leftarrow$ BERT(s_{des})
end for
for each mashup m, service s in mashup-service-tag network graph G **do**
  $vec_m \leftarrow$ MAGNN(G)
end for
$clusters \leftarrow \emptyset$
for each vec_m in batch of size N **do**
  $cluster \leftarrow \{vec_m\}$ grouped by (11) similarity
  $clusters$.add$(cluster)$
end for
$vec_{m'_{des}} \leftarrow$ BERT(m'_{des})
similar_list $= \emptyset$
for each mashup m **do**
  $sim_{m',m} = \frac{\vec{v}_{m'_{des}} \cdot \vec{v}_{m_{des}}}{\|\vec{v}_{m'_{des}}\| \cdot \|\vec{v}_{m_{des}}\|}$
  similar_list.add$(sim_{m',m})$
end for
preference_list $= \emptyset$
service_count $\leftarrow$ new Dictionary()
for each m in similar_list **do**
  $cluster \leftarrow$ findCluster$(m, clusters)$
  for each s in getServicesInCluster$(cluster)$ **do**
    service_count$[s] \leftarrow$ service_count.$get(s, 0) + 1$
  end for
end for
recommend_list $\leftarrow$ sortByValueDescending(service_count)
Return top-K (recommend_list)

V. Experiment

The ProgrammableWeb dataset is used in this section to report the empirical performance of FSSIM. It attempts to respond to the following research questions:

RQ1: How does FSSIM compare to the baseline models?

RQ2: Is the proposed method of combining structural and textual information effective?

RQ3: What impact do the model parameters have on the recommendation efficacy?

A. Experimental Settings

1) Dataset: ProgrammableWeb, the world's largest online web service registry, is often used as a dataset to validate web API recommendation systems. We crawl data from ProgrammableWeb up to June 2021 to form our dataset. As shown in Table I, the dataset is categorized into three groups based on the number of mashups that use each API. Most mashups typically utilize two or three services.

TABLE I
Dataset Statistics (After Preprocessing)

	Dataset	mashups	APIs	Average
All mashups	Dataset#1	5796	1792	1.84
mashups with at least 2 APIs	Dataset#2	2323	1241	3.12
mashups with at least 3 APIs	Dataset#3	1027	772	4.54

While the dataset used in this study was primarily crawled in 2021, it is important to note that ProgrammableWeb remains the most significant and widely adopted benchmark dataset in web API recommendation research. The website underwent migration after 2021, after which the original site ceased to be actively maintained and now only supports a limited number of popular web APIs and mashups. Nevertheless, research utilizing ProgrammableWeb data continues to be actively published and cited to this day, underscoring its enduring relevance as a domain

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PROGRAMMABLEWEB DATASET INFORMATION TABLE

Item	Value
The number of all mashup applications	5796
mashups with at least 2 APIs	2323
mashups with at least 3 APIs	1027
The number of all APIs	14312
Number of API call records contained	1792
The average number of APIs called per mashup	1.84
Total number of function tags for the mashup	734
Total number of tags shared between mashups and web APIs	479
Total number of function tags in the web API	1800

benchmark [6], [15], [18], [20]. The volume of data used in our study is comparable to that employed in these existing works and aligns with current research practices in the field.

Furthermore, we have incorporated specific information from the crawled dataset to demonstrate significant data sparsity. Specifically, out of 14,312 APIs collected, only 1,792 have call records. Further details are provided in Table II.

2) Evaluation Metrics: We conduct experiments on three segmented datasets, using 70% of each dataset's data for training, 15% for testing, and the remaining 15% for validation.

We evaluate performance using $Precision@k$, $Recall@k$, and $MAP@k$, which are standard metrics for assessing mashup recommendation systems [31], [32], [33].

As defined in (21), $Precision@k$ quantifies the proportion of recommended services (within the top $-k$ list, denoted as $\mathrm{top_m}(k)$) for a specific mashup m that match the actual API services invoked by m in the test data (denoted as $test_m$). The model's precision $Precision@k$ is calculated as the average accuracy of this precision value across all mashups.

Recall, denoted by $Recall@k$, measures the proportion of actual services in the testing data are recommended for a specific mashup, as defined in (22). The overall $recall$ of the model, represented by $Recall@k$, is computed as the average recall across all mashups in the test set.

The formula (23) describes an index called Mean Average Precision (MAP). $I(i)$ indicates whether the service at position i is an actual component API service of the target mashup. k_m denotes the number of component services associated with mashup m. k_i represents the number of component services of the mashup that appear within the top i positions of the recommendation list.

$$Precision@k = \sum_{\mathrm{m}=1}^{\mathrm{n}} \frac{|\mathrm{top_m}\,(\mathrm{k}) \cap \mathrm{test_m}|}{k} \tag{21}$$

$$Recall@k = \frac{1}{\mathrm{n}} \sum_{\mathrm{m}=1}^{n} \frac{|\mathrm{top_m}(\mathrm{k}) \cap \mathrm{test_m}|}{|\mathrm{test_m}|} \tag{22}$$

$$MAP@k = \frac{1}{n} \sum_{\mathrm{m}=1}^{n} \frac{1}{\mathrm{k_m}} \sum_{i=1}^{k} \left(\frac{k_i}{i} \times I(i) \right) \tag{23}$$

3) Comparison With Baseline Approaches: To validate the effectiveness of our model in both the functional semantic component and the structural information component, FSSIM was compared against six benchmarks. These include methods focusing primarily on content information, BERT, Doc2vec, and LDA; LightGCN, which centers on structural information derived from historical interactions; the typical SRMG method combining both content and structural information; and the Pop method, which is highly susceptible to data sparsity.

Pop: It begins by counting the number of times each API is invoked by mashups. It then sorts the APIs in descending order of their usage count. The most popular top $-k$ APIs form the final recommendation List.

LDA [34]: As described of mashups, representation vectors generated by LDA. Candidate mashups similar to the target mashups are then identified based on the cosine similarity of these vectors. The API services invoked by these candidate mashups are ranked by popularity to generate the recommendation list.

Doc2Vec [35]: This approach starts by generating phrase embeddings for service and mashup descriptions using the Doc2Vec tool. The top-ranked online services are then recommended based on their vector similarity to the target mashup.

LightGCN [36]: The word embeddings of mashup and service descriptions are first encoded using the BERT model. Next, user and item embeddings are learned using LightGCN by propagating them through the mashup-service-tag graph. Finally, the obtained embeddings are used for similarity computation to find the mashup most similar to a given target, which is then used to identify suitable web API services.

BERT [37]: It represents the word embeddings of mashups and service descriptions using the BERT tool. The top-ranked comparable online services are then suggested based on the closeness of their representation to the intended mashup. Some researchers have already used BERT for mashup recommendations and achieved good results [15], [38].

SMRG [15]: It first employs BERT to learn input feature, then utilizes GraphGAN [39] to generate embeddings. By computing cosine similarity between embeddings, it identifies top $-k$ mashups and uses collaborative filtering to recommend APIs based on the APIs selected by these mashups.

B. Overall Performance Comparison(RQ1)

For a fair comparison, we fix the candidate sequences of all methods to a consistent length. To ensure fairness, we fine-tune the parameters of all compared methods until these models reach their optimal performance level.

When comparing the effectiveness of various techniques, our proposed model outperforms all six baselines across all datasets. The specific results are detailed in Tables III – XI.

The performance of Pop on the used segmented datasets is the worst. This is likely because it relies solely on the frequency of API calls without incorporating specific semantic information. Additionally, Pop is highly susceptible to the long-tail effect, potentially generating identical recommendation lists for mashups with different functional requirements.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PRECISION PERFORMANCE ON DATASET#1(ALL MASHUPS)

	$Precision@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.3534	0.2284	0.1810	0.1521	0.1365
LDA	0.4034	0.2672	0.2074	0.1681	0.1403
Doc2vec	0.4896	0.3077	0.2270	0.1793	0.1544
BERT	0.5120	0.3137	0.2272	0.1883	0.1596
LightGCN	0.5379	0.3353	0.2540	0.2034	0.1689
SRMG	0.5603	0.3568	0.2724	0.2241	0.1893
FSSIM	**0.6621**	**0.4332**	**0.3253**	**0.2567**	**0.2132**

TABLE IV
RECALL PERFORMANCE ON DATASET#1(ALL MASHUPS)

	$Recall@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.2627	0.3073	0.3420	0.3699	0.4017
LDA	0.2951	0.3686	0.4145	0.4435	0.4567
Doc2vec	0.3672	0.4426	0.4711	0.4876	0.5097
BERT	0.3607	0.4439	0.4672	0.5000	0.5182
LightGCN	0.3956	0.4666	0.5117	0.5342	0.5478
SRMG	0.4060	0.4839	0.5348	0.5755	0.5947
FSSIM	**0.4934**	**0.5892**	**0.6339**	**0.6642**	**0.6728**

TABLE V
MAP PERFORMANCE ON DATASET#1(ALL MASHUPS)

	$MAP@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.2627	0.2803	0.2870	0.2906	0.2920
LDA	0.2951	0.3192	0.3247	0.3255	0.3257
Doc2vec	0.3672	0.3870	0.3921	0.3932	0.3940
BERT	0.3791	0.4015	0.4054	0.4078	0.4090
LightGCN	0.3956	0.4224	0.4277	0.4295	0.4307
SRMG	0.4060	0.4413	0.4480	0.4512	0.4532
FSSIM	**0.4934**	**0.5432**	**0.5466**	**0.5489**	**0.5497**

TABLE VI
PRECISION PERFORMANCE ON DATASET#2(MASHUPS WITH AT LEAST 2 APIS)

	$Precision@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.3541	0.2833	0.2458	0.2156	0.2016
LDA	0.4166	0.3187	0.2652	0.2177	0.1858
Doc2vec	0.5291	0.3979	0.3055	0.2562	0.2183
BERT	0.5500	0.3812	0.2861	0.2500	0.2191
LightGCN	0.5708	0.4041	0.3277	0.2729	0.2291
SRMG	0.6041	0.4437	0.3597	0.3041	0.2633
FSSIM	**0.6856**	**0.5239**	**0.4208**	**0.3537**	**0.3007**

TABLE VII
RECALL PERFORMANCE ON DATASET#2(MASHUPS WITH AT LEAST 2 APIS)

	$Recall@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.1348	0.2052	0.2518	0.2858	0.3291
LDA	0.1549	0.2367	0.2934	0.3176	0.3370
Doc2vec	0.1976	0.2940	0.3299	0.3600	0.3811
BERT	0.2127	0.2888	0.3166	0.3609	0.3879
LightGCN	0.2205	0.3040	0.3624	0.3984	0.4165
SRMG	0.2313	0.3320	0.3967	0.4409	0.4664
FSSIM	**0.2689**	**0.4212**	**0.4823**	**0.4998**	**0.5304**

TABLE VIII
MAP PERFORMANCE ON DATASET#2(MASHUPS WITH AT LEAST 2 APIS)

	$MAP@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.1348	0.1774	0.1936	0.2024	0.2058
LDA	0.1549	0.2130	0.2264	0.2283	0.2290
Doc2vec	0.1976	0.2687	0.2782	0.2861	0.2874
BERT	0.2127	0.2739	0.2812	0.2891	0.2926
LightGCN	0.2205	0.2861	0.3003	0.3048	0.3057
SRMG	0.2313	0.3166	0.3327	0.3405	0.3454
FSSIM	**0.2689**	**0.3906**	**0.3987**	**0.3993**	**0.4020**

TABLE IX
PRECISION PERFORMANCE ON DATASET#3(MASHUPS WITH AT LEAST 3 APIS)

	$Precision@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.3980	0.3592	0.3430	0.3155	0.3009
LDA	0.4951	0.3640	0.3139	0.2645	0.231
Doc2vec	0.5825	0.4417	0.3398	0.2839	0.2504
BERT	0.5728	0.4174	0.3268	0.2985	0.2737
LightGCN	0.6019	0.466	0.3915	0.3349	0.2854
SRMG	0.6504	0.5097	0.4239	0.3689	0.3339
FSSIM	**0.7238**	**0.6565**	**0.4993**	**0.4537**	**0.3908**

TABLE X
RECALL PERFORMANCE ON DATASET#3(MASHUPS WITH AT LEAST 3 APIS)

	$Recall@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.1006	0.1772	0.2421	0.2922	0.3446
LDA	0.1232	0.1729	0.2275	0.2547	0.2804
Doc2vec	0.1426	0.2084	0.2401	0.2607	0.2889
BERT	0.1414	0.2020	0.2280	0.2730	0.3117
LightGCN	0.1497	0.2328	0.2862	0.3264	0.3493
SRMG	0.1604	0.2495	0.3032	0.3479	0.3878
FSSIM	**0.1893**	**0.2987**	**0.366**	**0.4184**	**0.4511**

In contrast, text-feature mining techniques LDA, BERT, and Doc2vec performed better. The effectiveness was positively correlated with their text encoding capabilities. BERT outperformed both Doc2vec and the topic model LDA when processing paragraph text, contributing to its superior results across different segmented datasets. Fruthermore, higher vector dimensions enhance feature representation. Due to BERT's advanced bidirectional encoding structure and its 768-dimensional feature vector (whereas Doc2vec's 40 dimensions), BERT demonstrated performance improvements over LDA and Doc2vec on all datasets. For example, on dataset#1, the precision of BERT was at least 20% higher than LDA, and at least 15% higher than Doc2vec, clearly highlighting BERT's advantage in capturing functional semantic information. Consequently, FSSIM

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
MAP PERFORMANCE ON DATASET#3(MASHUPS WITH AT LEAST 3 APIS)

	$MAP@K$				
Baseline	$@K=1$	$@K=2$	$@K=3$	$@K=4$	$@K=5$
Pop	0.1006	0.1488	0.1867	0.2071	0.2151
LDA	0.1232	0.1615	0.1927	0.1971	0.1986
Doc2vec	0.1426	0.1929	0.2145	0.2235	0.2267
BERT	0.1414	0.1918	0.2088	0.2271	0.2352
LightGCN	0.1497	0.2152	0.2484	0.2588	0.2610
SRMG	0.1604	0.2354	0.2730	0.2911	0.3024
FSSIM	**0.1893**	**0.3142**	**0.3473**	**0.3634**	**0.3699**

also incorporates a BERT-based functional semantic encoding component.

Both LightGCN and SRMG utilize BERT to enhance semantic information extraction. However, they employed different strategies for mining structural information. LightGCN aggregates neighbor information, while SRMG uses GraphGAN to mine structural information. Compared to LightGCN's simpler approach, SRMG achieves better performance, indicating the important of structural information in web API recommendations. SRMG obtained excellent second-place performance across all segmented datasets and evaluation metrics.

Compared to the aforementioned methods, FSSIM demonstrated superior performance across all datasets. Compared to the best-performing baseline model SRMG, FSSIM yielded at least a 10.7% increase in precision, a 15.7% increase in recall, and a 19.2% increase in MAP across the different segmented datasets.

Summary: FSSIM demonstrates clear superiority over all baselines in web API recommendation across all datasets and metrics. This advantage is attributed to its combination of structural information mining, which aggregates neighbor features by learning distinct weights for nodes and metapaths and BERT-based semantic encoding.

C. Ablation Analysis of FSSIM Components(RQ2)

To investigate the impact of the structural information incorporated in our method on the accuracy of recommendations, we conduct performance testing on BERT, FSSIM, and Structural Information Component (SIC) respectively. As the core part of the semantic component proposed in this paper, BERT is used alone to verify the influence of structural information on recommendation performance. We primarily compare the requirements document provided by developers with the functional descriptions of existing mashups, select the group of mashups with the highest similarity to it, and make recommendations based on the APIs they call. SIC represents the structural semantic component in this paper, constructed based on a heterogeneous graph of mashup-tag-API relationships. Here, instead of encoding the functional descriptions of mashups and web APIs as initial feature inputs, we classify mashups using only historical interactions and structural information derived from common tags. Then, by comparing the requirements document provided by developers with the functional documentation of existing mashups in the identified cluster, we select the most similar cluster of web APIs for recommendations. FSSIM is the complete model proposed in this paper, which integrates the textual semantic component and the structural semantic component for recommendations. We conduct experiments evaluating different components of the model on dataset #1 (complete dataset), measuring MAP and recall, to observe the impact of structural information on recommendation performance.

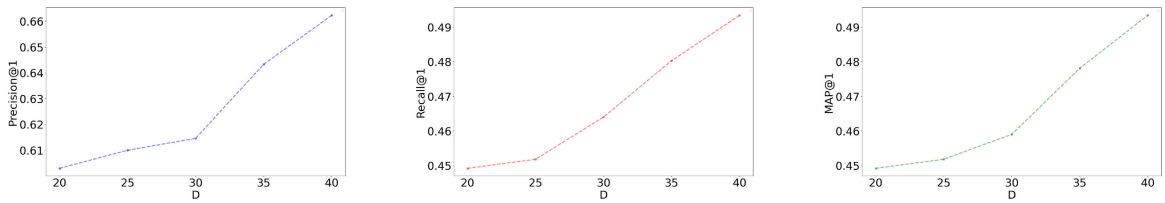
In Fig. 7, we evaluate the performance of three methods under varying K conditions. It can be observed that our model exhibits varying degrees of improvement across different K conditions. This effectively demonstrates that integrating structural information with textual information significantly enhances the performance of recommendation system, as well as the feasibility of our proposed method. Among these models, the BERT model, which relies solely on textual information, achieves the worst results. This is because the functionalities of mashups and web APIs often operate on different dimension; mashup descriptions typically represent high-dimensional functions composed of multiple features, which also indicates that directly applying the functional overlay method of web APIs while ignoring the semantic gap between mashups and web APIs severely impacts recommendation performance. The SIC model ranks second, with its advantage over traditional collaborative filtering lies in capturing higher-order structural information beyond simple collaborative signals. The multi-path aggregation method effectively captures the contributions of different node types to the target node's relationships. The FSSIM, which combines functional semantic components and structural semantic components, achieves superior performance. Specifically, MAP improving by 34% and 11% compared to BERT and SIC, respectively, while Recall improving by 27% and 12% . This improvement stems from FSSIM's comprehensive consideration of data sparsity during structural information extraction. It alleviates sparsity through similar functional labels and enhances the weight of structural information under specific paths using metapaths. These results indicate that the structural information extracted based on our proposed network structure plays a positive role in recommendations.

Summary: Based on experiments comparing BERT, SIC, and FSSIM, FSSIM outperforms BERT and SIC respectively. This demonstrates the positive impact of structural semantics derived from mashup-tag-API relationships on alleviating data sparsity and enhancing recommendations.

D. Parameter impact(RQ3)

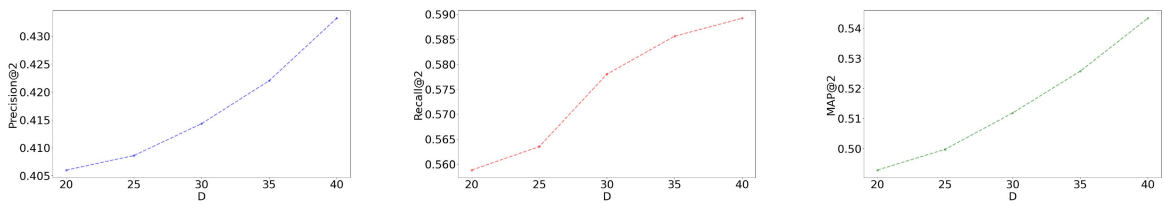
For FSSIM, two key factors affect recommendation. One is the size of the search neighbors, which determines the generalization of the entire recommendation list. The other is the embedding dimension, which directly affects the representation power of the mashup embeddings. To ascertain the effects of the search neighbor size and the embedding dimension on the recommendation results, we conduct two experiments on the dataset.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.



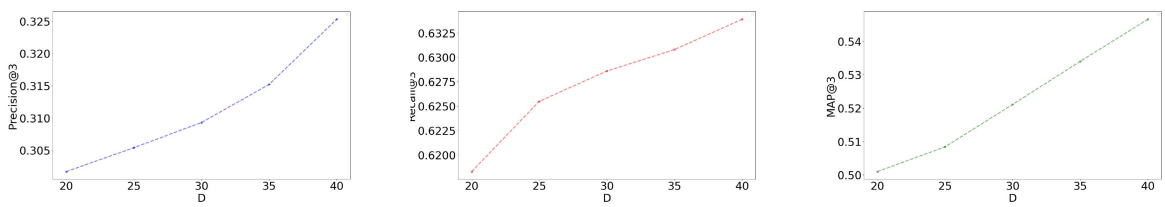
(a) Precision (b) Recall (c) MAP

Fig. 3. Precision,Recall and MAP in different embedding dimensions when K=1.



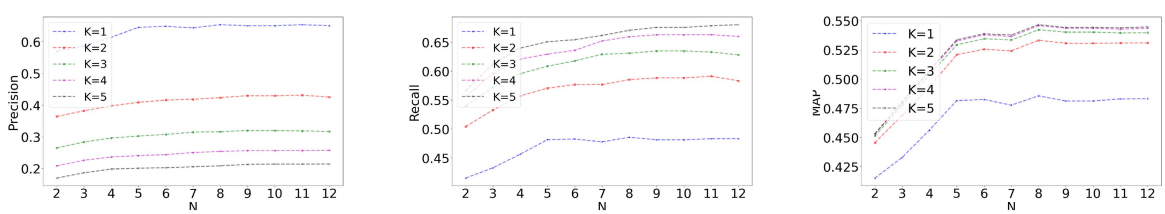
(a) Precision (b) Recall (c) MAP

Fig. 4. Precision,Recall and MAP in different embedding dimensions when K=2.



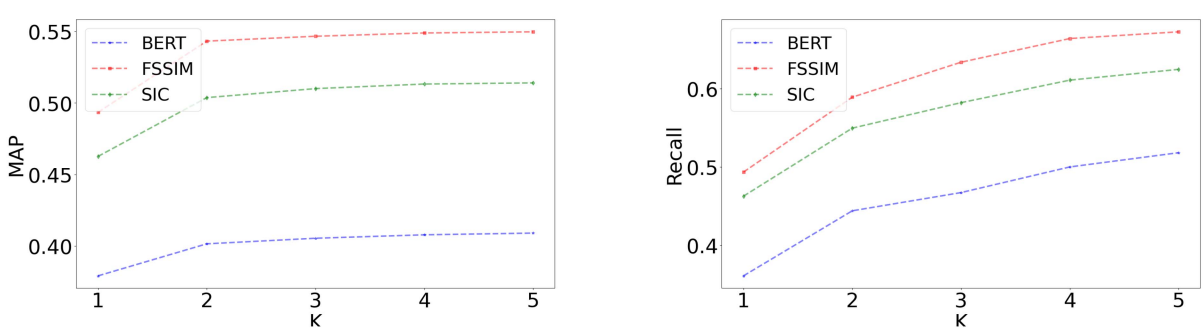
(a) Precision (b) Recall (c) MAP

Fig. 5. Precision,Recall and MAP in different embedding dimensions when K=3.



(a) Precision (b) Recall (c) MAP

Fig. 6. Precision,Recall and MAP in different size of the search neighbors.



(a) MAP (b) Recall

Fig. 7. Ablation study on dataset#1(all mashups).

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

We test the impact of the dimension of the embedded vector on model performance by changing $d = 20, 25, 30, 35, 40$. Because the number of candidate APIs for a mashup typically does not exceed 3, we observe the impact on $Precision@K$, $Recall@K$, $MAP@K$ at $K = 1, 2, 3$. The impact is shown in Figs. 3, 4, and 5.

The specific results indicate that as the embedding dimension D increases, the performance scores of the recommendation system also increase, which aligns with our expectations. Notably, at $D = 40$, certain performance metrics decrease. We speculate that this may be due to overfitting caused by excessive model complexity.

In addition, we also investigate the impact of the number of search neighbors N on mashup experimental results following similarity computation. As shown in Fig. 6, we test N values ranging from 2 to 12. To a certain extent, FSSIM's performance improves as N increases. However, once N exceeds a critical point, its performance begins to deteriorate. This may occur because the neighbor interaction learning process introduces noisy data. Specifically, an overly broad search range includes low-similarity mashups in the candidate list, degrading recommendation performance.

In Fig. 7, we can conclude that as N gradually increases from 2, the performance of the recommendation system also improves. This is because too small N does not search for enough samples. When N increases, more similar samples are included in the candidate sequence, which makes the aggregated service recommendations more reliable. But when N reaches the critical value, it means that the recommendation system will include mashups that are less similar in the candidate list, which will lead to a decrease in recommendation performance. In our system, we set $N = 9$. This is because we select the top 3 groups in the $mashup_{set}$, and group three mashups per group based on their structural characteristics, resulting in a final list length N of $3 \times 3 = 9$).

Summary: Both embedding dimension d and search neighbor size N significantly impact FSSIM's recommendation performance. Optimal values exist for both parameters: performance increases with d until potential overfitting occurs, and increases with N until noisy neighbors degrade results.

VI. Conclusions and Future Work

This article proposes a model named FSSIM. This model comprises two core components: the functional semantic component and the structural information component. The functional semantic component, built upon BERT, primarily captures the demand characteristics of service mashups. Conversely, the structural information component leverages metapaths Aggregated graph neural networks for feature mining. We propose a recommendation method that integrates both structural and functional semantics. First, to locate relevant service mashups using functional semantic information; then, to find groups of mashups with similar selections based on structural similarity; and finally, to recommend set of web APIs based on historical invocation information within those groups. To implement this method, we construct a structured service network to mine high-quality features and perform grouping. Subsequently, we identify the most relevant group based on demand characteristics and recommend a series of highly compatible web API combinations within that group. Additionally, we design a multi-path aggregation method to mine structural information. Specifically, we enhance structural information extraction by constructing a heterogeneous graph neural network for metapath aggregation. This network dynamically adjusts the contribution weights of different neighboring nodes to the central node according to different metapaths, thereby mining semantically specific structural information. We evaluate the performance of our method by comparing the direct use of embeddings and functional semantic information with our proposed joint utilization method. Experimental studies conducted on the real ProgrammableWeb dataset show that FSSIM outperforms various advance recommendation techniques in key metrics, such as precision, recall, and MAP, demonstrating the superiority of our method.

In the future, research on web service recommendation systems can be further advanced in the following directions. Dynamic metapaths and higher-order heterogeneous relationship mining: While current feature aggregation mechanisms based on predefined metapaths can capture certain structural features, they struggle to adapt to complex and dynamic heterogeneous service networks. Future work should explore dynamic metapath generation strategies. These can leverage graph attention mechanisms or reinforcement learning techniques to dynamically identify key metapaths. Additionally, incorporating hypergraph modeling technology can help capture higher-order interactive relationships among mashups, APIs, and tags. For instance, regarding the service combination patterns across functional tags, hyper-edge modeling incorporating multi-layer semantic dependencies can be designed to uncover potential functional synergy patterns. Compatibility modeling and optimization in complex scenarios: This area needs to move beyond the limitations of existing binary combination analysis. Future research should focus on developing compatibility inference mechanisms for API compositions. This can be achieved by constructing a compatibility prediction model based on multi-order interaction graphs, utilizing graph neural networks and combinatorial optimization algorithms (such as Monte Carlo tree search). Furthermore, it is crucial to integrate dynamic factors like QoS fluctuations and API version updates, establishing a hierarchical compatibility evaluation framework.

References

[1] X. Liu, Y. Hui, W. Sun, and H. Liang, "Towards service composition based on mashup," in *Proc. 2007 IEEE Congr. on Serv.*, 2007, pp. 332–339.

[2] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.

[3] B. Cao, X. F. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIs recommendation for mashup development," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

[4] Y. Hao, Y. Fan, W. Tan, and J. Zhang, "Service recommendation based on targeted reconstruction of service descriptions," in *Proc. 2017 IEEE Int. Conf. Web Serv.*, 2017, pp. 285–292.

[5] L. Qi, H. Song, X. Zhang, G. Srivastava, X. Xu, and S. Yu, "Compatibility-aware web API recommendation for mashup creation via textual description mining," *ACM Trans. Multimidia Comput. Commun. Appl.*, vol. 17, no. 1s, pp. 1–19, 2021.

[6] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with API co-invocations," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 502–515, Mar./Apr. 2021.

[7] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[8] Z. Chen, Y. Sun, D. You, F. Li, and L. Shen, "An accurate and efficient web service QoS prediction model with wide-range awareness," *Future Gener. Comput. Syst.*, vol. 109, pp. 275–292, 2020.

[9] J. Li, J. Wang, Q. Sun, and A. Zhou, "Temporal influences-aware collaborative filtering for QoS-based service recommendation," in *Proc. 2017 IEEE Int. Conf. Serv. Comput.*, 2017, pp. 471–474.

[10] H. Wu, K. Yue, B. Li, B. Zhang, and C.-H. Hsu, "Collaborative QoS prediction with context-sensitive matrix factorization," *Future Gener. Comput. Syst.*, vol. 82, pp. 669–678, 2018.

[11] G. Zou, M. Jiang, S. Niu, H. Wu, S. Pang, and Y. Gan, "QoS-aware web service recommendation with reinforced collaborative filtering," in *Proc. 16th Int. Conf. Service-Oriented Comput.*, Hangzhou, China, Springer, 12–15, 2018, pp. 430–445.

[12] S. Lian and M. Tang, "API recommendation for mashup creation based on neural graph collaborative filtering," *Connection Sci.*, vol. 34, no. 1, pp. 124–138, 2022.

[13] Q. Qi, J. Cao, and Y. Liu, "API-program: An API package recommendation model based on the graph representation learning method," in *Proc. 19th Int. Conf. Service-Oriented Comput.*, Springer, Nov. 22–25, 2021, pp. 859–866.

[14] Y. Xiao et al., "Structure reinforcing and attribute weakening network based API recommendation approach for mashup creation," in *Proc. 2020 IEEE Int. Conf. Web Serv.*, 2020, pp. 541–548.

[15] T. Yu, D. Yu, D. Wang, and X. Hu, "Web service recommendation for mashup creation based on graph network," *J. Supercomput.*, vol. 79, no. 8, pp. 8993–9020, 2023.

[16] X. Wang, M. Xi, and J. Yin, "Functional and structural fusion based web API recommendations in heterogeneous networks," in *Proc. 2023 IEEE Int. Conf. Web Serv.*, 2023, pp. 91–96.

[17] T. Liang, L. Chen, J. Wu, H. Dong, and A. Bouguettaya, "Meta-path based service recommendation in heterogeneous information networks," in *Proc. 14th Int. Conf. Service-Oriented Comput.*, Banff, AB, Canada, Springer, Oct. 10–13, 2016, pp. 371–386.

[18] F. Xie, S. Li, L. Chen, Y. Xu, and Z. Zheng, "Generative adversarial network based service recommendation in heterogeneous information networks," in *Proc. 2019 IEEE Int. Conf. Web Serv.*, 2019, pp. 265–272.

[19] Y. Zhang, H. Yang, and L. Kuang, "A web API recommendation method with composition relationship based on GCN," in *Proc. 2020 IEEE Int. Conf. Parallel Distrib. Process. Appl. Big Data Cloud Comput. Sustain. Comput. Commun. Social Comput. Netw.*, 2020, pp. 601–608.

[20] X. Hu, "Biased random walk based web API recommendation in heterogeneous network," in *Proc. 2024 IEEE Int. Conf. Web Serv.*, 2024, pp. 172–177.

[21] Z. Gao et al., "SeCo-LDA: Mining service co-occurrence topics for recommendation," in *Proc. 2016 IEEE Int. Conf. Web Serv.*, 2016, pp. 25–32.

[22] Y. Wu, F. Xie, L. Chen, C. Chen, and Z. Zheng, "An embedding based factorization machine approach for web service QoS prediction," in *PRoc. 15th Int. Conf. Service-Oriented Comput.*, Malaga, Spain, Springer, Nov. 13–16, 2017, pp. 272–286.

[23] X. Wang, X. Liu, J. Liu, X. Chen, and H. Wu, "A novel knowledge graph embedding based API recommendation method for mashup development," *World Wide Web*, vol. 24, no. 3, pp. 869–894, 2021.

[24] Y. Wang, S. Tang, Y. Lei, W. Song, S. Wang, and M. Zhang, "DisenHAN: Disentangled heterogeneous graph attention network for recommendation," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, New York, NY, USA, 2020, pp. 1605–1614.

[25] Y. Pang et al., "Heterogeneous global graph neural networks for personalized session-based recommendation," in *Proc. 15th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA, 2022, pp. 775–783.

[26] X. Fu, J. Zhang, Z. Meng, and I. King, "MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding," in *Proc. Web Conf.*, 2020, pp. 2331–2341.

[27] S. Fan et al., "Metapath-guided heterogeneous graph neural network for intent recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2019, pp. 2478–2486.

[28] X. Wang et al., "SEHGN: Semantic-enhanced heterogeneous graph network for web API recommendation," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2836–2849, Sep./Oct. 2024.

[29] X. Li, X. Zhang, P. Wang, and Z. Cao, "Web services recommendation based on metapath-guided graph attention network," *J. Supercomput.*, vol. 78, no. 10, pp. 12621–12647, 2022.

[30] C. Sang, X. Deng, and S. Liao, "Mashup-oriented web API recommendation via full-text semantic mining of developer requirements," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2755–2768, Jul./Aug. 2023.

[31] Q. Gu, J. Cao, and Y. Liu, "CSBR: A compositional semantics-based service bundle recommendation approach for mashup development," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3170–3183, Nov./Dec. 2022.

[32] B. Cao, L. Zhang, M. Peng, Y. Qing, G. Kang, and J. Liu, "Web service recommendation via combining bilinear graph representation and xDeepFM quality prediction," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1078–1092, Jun. 2023.

[33] Y. Hao, Y. Fan, W. Tan, and J. Zhang, "Service recommendation based on targeted reconstruction of service descriptions," in *Proc. 2017 IEEE Int. Conf. Web Serv.*, 2017, pp. 285–292.

[34] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, no. Jan, pp. 993–1022, 2003.

[35] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2014, pp. 1188–1196.

[36] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 639–648.

[37] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv: 1810.04805*.

[38] V. G. Lad, E. Lima, and X. Liu, "HSG-CDM: A heterogeneous service graph contextual deep model for web service classification," in *Proc. 2022 IEEE Int. Conf. Serv. Comput.*, 2022, pp. 104–113.

[39] H. Li, J. Zhang, L. Yang, J. Qi, and Z. Gao, "Graph-GAN: A spatial-temporal neural network for short-term passenger flow prediction in urban rail transit systems," 2022, *arXiv.2202.06727*.

Chunyan Sang received the PhD degree in control theory and control engineering from Chongqing University, Chongqing, China, in 2014. She is currently an associate professor with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications. Her research interests include service recommendation, data mining, and its applications.

Yahao Liu received the BS degree in 2022. He is currently working toward the MS degree with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications. His research interests include service discovery and recommendation.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.

Shigen Liao received the PhD degree in management science and engineering from Chongqing University, Chongqing, China, in 2021. He is currently an assistant engineer with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications. His research interests include intelligent manufacturing, knowledge service, and quality management.

Junhao Wen received the PhD degree from Chongqing University, Chongqing, China, in 2008. He is currently a professor with the College of Big Data and Software Engineering, Chongqing University. His research interests include service computing, cloud computing, and software dependable engineering. He has authored more than 80 refereed journal and conference articles in the above areas. He holds more than 30 research and industrial grants and developed many commercial systems and software tools.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:24:24 UTC from IEEE Xplore. Restrictions apply.